https://cloud.notewise.dev/note/lvfRNuDtov4nwYJACM8-vjDV?page=vBHO4PIZ_nMOrVSVzN2SLjJO

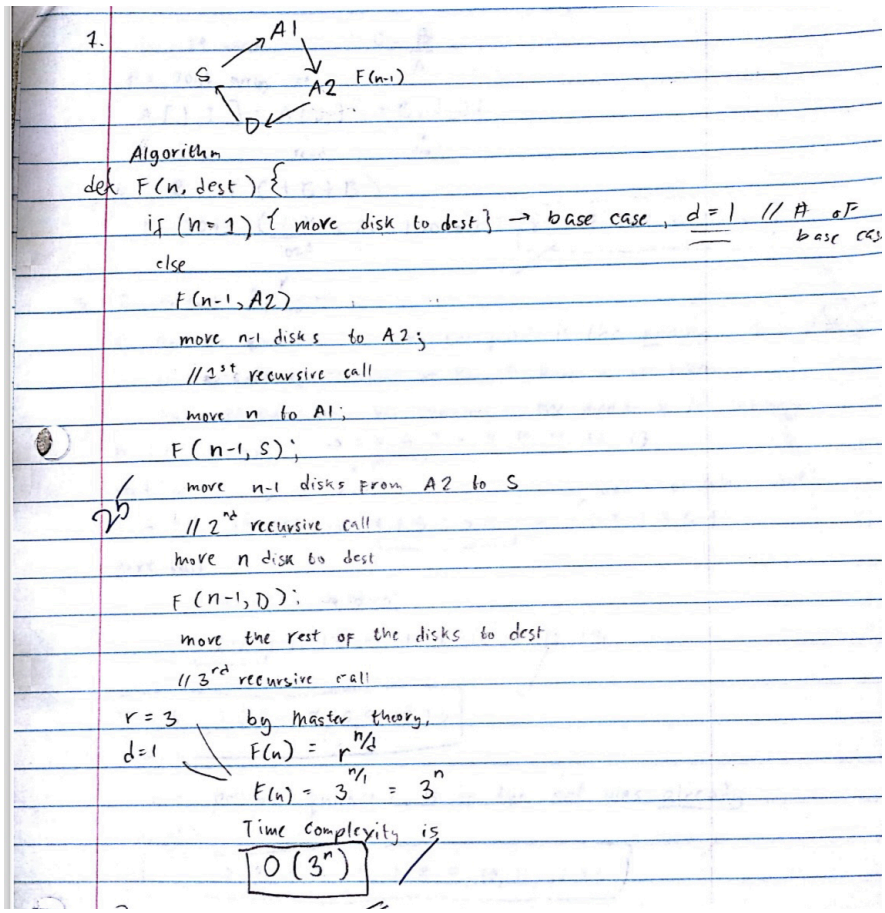
xe

Problem 1

⅗ times is a hanoi problem, ⅕ times a rectangular matrix problem, ⅕ times a page transfer problem

2008

Design a function (algorithm) that solves the Towers of Hanoi game for the following graph G=(V,E) with V=(Start, Dest, Aux1, Aux2} and E = {(Start, Dest), (Dest, Aux1). (Ausl, Aux2), (Aux2, Start);. Estimate the time complexity of your function, in terms of the number n of disks to be moved.



2012

1. Design a function (algorithm) that solves the Towers of Hanoi game for the following graph G=(V,E) with V={Start, Aux1, Dest, Aux2} and E = {(Start, Aux1), (Aux1, Aux2), (Aux2, Dest), (Dest, Aux1)}. Estimate the time complexity of your function, in terms of the number n of disks to be moved. **For the drawing below, Dest goes to A1 not Start**

Pseudo Code:

H1(d):

if(d == 1):

Move disk from start to dest

Else:

H1(d-1)

Move disk d from start to a2

H2(3, 1, d-1) // moves stack of d-1 disks at dest to a1

// move disk d from a2 to dest

H2(1, 3, d-1) // moves stack of d-1 disks at a1 to dest

H2(int st, int ed, int disks)

if(d == 1):

// move d1 from st to end

Else:

H2(st, st+2,. Disks - 1)

// move disk d forward once

If (st + 1 == ed) {

H2(st + 2, ed, d-1)

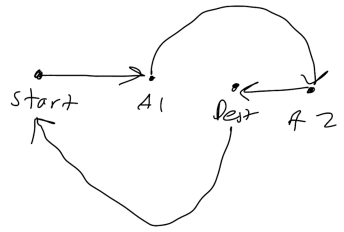
Return;

}

H2;(st + 2, ed,, d-1)

// move disk d from st + 1 to ed

H2(st, ed, d-1)



2018

Design a function (algorithm) that solves the Towers of Hanoi game for the following graph G=(V,E) with V={Start, Aux 1, Aux2, Dest } and E = {(Start, Aux 1), (Aux 1, Aux2), (Aux2, Dest), (Dest, Start) }. Estimate the time complexity of your function, in terms of the number n of disks to be moved. **Think of it as a square.**

Pseudo Code:

H(int st, int ed, int disks)

If (disks == 1):

// move the disk from start to end

Else:

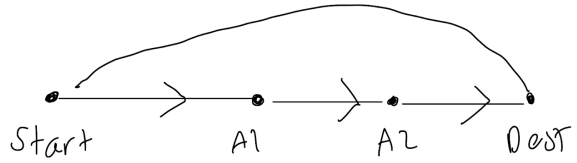
H(st, st+2, disks - 1) // move stack above forward twice

// move disk forward once

H(st+2, st, disks - 1) // move stack above back to start

// finishing moving disk to end

H(st, ed, disks - 1) // move rest of stack to above the disk



2020

Consider the problem of multiplying n rectangular matrices discussed in class. Assume, in contrast to what we did in class, that we want to determine the maximum number of scalar multiplications that one might need (that is, compute the maximum of all possible parenthesizations).
Formulate precisely an algorithm that determines this value. Then carry out your method on the following product to show what is the worst-possible parenthesization and how many scalar multiplications are required to carry it out:
M9,1*M1,8*M8,2*M2,10*M10,4.

Definitions:

- Let there be n matrices M1,M2,…,Mn
- Each matrix Mi has dimensions pi−1×pi.
- Define a 2D table m[i][j] to store the maximum cost (number of scalar multiplications) needed to compute the product Mi×Mi+1×…×Mj.
- Define a 2D table s[i][j]s[i][j]s[i][j] to store the index kkk where the optimal split occurs between MiM_iMi and MjM_jMj.

Algorithm Steps:

1. **Initialization:**
    - For all i, set m[i][i]=0 because multiplying one matrix requires zero scalar multiplications.
2. **Compute Maximum Costs:**
    - For chain lengths l from 2 to n:
        - For i=1 to n−l + 1:
            - Set j=i+l−1
            - Initialize m[i][j]=−∞
            - For k=i to j−1:
                - Compute the cost q=m[i][k] + m[k+1][j] + p_{i-1}×pk×pj
                - If q>m[i][j], update:
                    - m[i][j]=q
                    - s[i][j]=k.
3. **Construct Optimal Parenthesization (Optional):**
    - Use the table s to reconstruct the parenthesization that results in the maximum cost.

Problem 2

⅘ times is a page transfer question, ⅕ times is a quicksort problem

HOW TO SOLVE ANY PAGE TRANSFER QUESTION

A= word count of each page
B = array length
C = B/A

#	Array Formula	Column Major	Row Major

1	A[i,j] = A[i,j] * B[i,j}	3 * B * B	3 * B/A * B
2	A[i,j] = A[i,j] * B[j,i]	B * (B + B +C)	B * (C + C + B)
3	A[i,j] = A[j,i] * B[j,i]	B * (C + C + B)	B * (C + B + B)
4	A[j,i] = A[j,i] * B[i,j]	B * (C + C + B)	B * (C + B + B)
5	A[j,i] = A[i,j] * B[i,j]	B * (B + B + C)	B * (C + C + B)
6	A[j,i] = A[j, i] * B[j, i]		B * (B + B +B)

2021

2. Consider QuickSort on the array A[1:n] and assume that the pivot element x (used to split the array A[lo:hi] into two portions such that all elements in the left portion A[lo:m] are ≤x and all elements in the right portion A[m:hi] are ≥x) is the **penultimate** element of the array to be split (i. e., A[hi - 1]).
For a specific value of n at least 13 (**your** choice), construct an assignment of the numbers 1...n to the n array elements that causes QuickSort, with the stated choice of pivot, to
(a) execute optimally (that is A[lo:m] and A[m:hi] are always of equal size)
(b) execute in the slowest possible way.

2022

Consider QuickSort on the array A[I:n] and assume that the pivot element x (used to split the array A[lo:hi] into two portions such that all elements in the left portion A[lo:m] are S* and all elements in the right portion A[m:hi] are >x) is the first element of the array to be split (i. e., Alo]).
For a specific value of n at least 13 (your choice), construct an assignment of the numbers
1..n to the n array elements that causes QuickSort, with the stated choice of pivot, to

- (a) execute optimally (that is A[lo:m] and A[m:hi] are always of equal size)
- (b) execute in the slowest possible way.

(a) Optimal Case:

In the optimal case, QuickSort achieves its best time complexity when the pivot divides the array into two equal (or nearly equal) subarrays at each step. This way, the algorithm will partition the array in a balanced manner, resulting in a time complexity of O(nlogn)O(n \log n)O(nlogn).

To achieve this, we need to ensure that the penultimate element, chosen as the pivot, is always such that it splits the array into two nearly equal halves. For n=13n = 13n=13, the ideal assignment is one where, at each partition step, the pivot ends up at the middle (or near the middle) of the array, resulting in a balanced recursion tree.

Here's a possible optimal input for n=13n = 13n=13:

Optimal Input:

A=[1,2,3,4,5,6,7,8,9,10,11,12,13]A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]A=[1,2,3,4,5,6,7,8,9,10,11,12,13]

Explanation:

- At the first step, the pivot is the penultimate element, which is A[11]=12A[11] = 12A[11]=12.
- This partitions the array into two parts: A[0:10]A[0:10]A[0:10] (elements less than 12) and A[12:12]A[12:12]A[12:12] (empty or single element).
- The left side is then partitioned recursively, with each subsequent pivot dividing the array in an almost balanced manner. The pivot always falls near the middle of the subarray, leading to an efficient partition and the optimal time complexity.

(b) Worst Case:

In the worst-case scenario, QuickSort performs poorly when each partition results in highly unbalanced subarrays. This leads to the worst-case time complexity of O(n2)O(n^2)O(n2). To achieve this, we must ensure that the penultimate element, chosen as the pivot, is either the largest or the smallest element in the subarray, so that one partition contains almost all the elements, and the other partition is nearly empty.

To force this behavior for n=13n = 13n=13, we can assign the elements in decreasing order. This way, each pivot will always be the largest element in the subarray, leading to highly unbalanced partitions.

Worst Case Input:

A=[13,12,11,10,9,8,7,6,5,4,3,2,1]A = [13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]A=[13,12,11,10,9,8,7,6,5,4,3,2,1]

Explanation:

- At the first step, the pivot is A[11]=2A[11] = 2A[11]=2, and it divides the array into two parts: A[0:10]A[0:10]A[0:10] (all larger than 2) and A[12:12]A[12:12]A[12:12] (single element, 1).
- The left part A[0:10]A[0:10]A[0:10] is then partitioned again with A[9]=3A[9] = 3A[9]=3 as the pivot, and the process continues.
- This pattern leads to highly unbalanced partitions (one large subarray and one tiny subarray at each step), resulting in the worst-case time complexity.

In summary:

- **Optimal input for n=13n = 13n=13**: [1,2,3,4,5,6,7,8,9,10,11,12,13][1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13][1,2,3,4,5,6,7,8,9,10,11,12,13]
- **Worst-case input for n=13n = 13n=13**: [13,12,11,10,9,8,7,6,5,4,3,2,1][13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1][13,12,11,10,9,8,7,6,5,4,3,2,1]

Seiun's Crappy Attempt to Solve (a)
Let n = 13.
QuickSort splits the array into halves depending on what's greater/smaller than the pivot value
For QuickSort to execute optimally, the pivot element must always be the midpoint of the current array portion, such that A[lo:m] and A[m:hi] are of equal length
The pivot is always the first element in the array

Array size 1: 13
Array size 2: 6

Array size 3: 3/4
Array size 4: 1/2
Array size 5: 1
[**x x x** x x **x** x **x x** x x **x** x]
[**x** x x x **x** x] [**x** x x x **x** x]
[**x** x x] [**x** x] [**x** x x] [**x** x]
[x] [x] [x] [x] [x] [x]
Items 1, 2, 3, 6, 8, 9, 12 are pivots
1 - midpoint of whole array

[**7** 4 2 3 5 6 1 11 9 10 8 12 13]
[**4** 2 3 5 6 1] [**11** 9 10 8 12 13]
[**2** 3 1] [5 6] [**9** 10 8] [12 13]

Hypothetical Assignment: [7 5 2 3 4 6 1 11 9 10 8 12 13]

Seiun's Crappy Attempt to Solve (b)
Let n = 13.
QuickSort splits the array into halves depending on what's greater/smaller than the pivot value
For QuickSort to execute in the worst case, the pivot element must always be the greatest or smallest of the current array portion, such that either A[lo:m] or A[m:hi] is the same length as A[lo:hi] and the other is 0
The pivot is always the first element in the array
We can easily do this by making the array already sorted

Hypothetical Assignment: [1 2 3 4 5 6 7 8 9 10 11 12 13]

Problem 3
⅗ times is a quicksort problem, ⅖ times is a rectangular matrix problem

HOW TO SOLVE QUICKSORT

Understanding Quicksort

- Quicksort uses a pivot element to divide an array into two subarrays such that
  - Left subarray has elements less than or equal to the pivot
  - Right subarray has elements greater than or equal to the pivot
- Here, the pivot is the first element of the current subarray

now set up worst case and best case

a) best case

6, 1, 2, 3, 4, 5, 7, 8, 9, 10, 11
arranged like this
sorts to
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

b) worst case if set is already sorted
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

2022

Consider the problem of multiplying n rectangular matrices discussed in class. Assume, in contrast to what we did in class, that we want to determine the maximum number of scalar multiplications that one might need (that is, compute the maximum of all possible parenthesizations).
Formulate precisely an algorithm that determines this value. Then carry out your method on the following product to show what is the worst-possible parenthesization and how many scalar multiplications are required to carry it out: M9,1 *M1,10*M10,2*M2,11 *M11,4.

Seiun's Crappy Attempt to Solve That Needs Double Checking Please
Points Given: (9,1), (1,10), (10,2), (2,11), (11,4)

Matrix:

	1	**2**	**3**	**4**	**5**
1	0	90	270	1300	1696
2		0	20	330	700
3			0	220	660
4				0	88
5					0

Fill in the diagonals (aka [1,1], [2,2], [3,3], [4,4], etc) with 0

Initial Values

M_1 = [a,b] = [9,1]
M_2 = [b,c] = [1,10]
M_3 = [c,d] = [10,2]
M_4 = [d,e] = [2,11]
M_5 = [e,f] = [11,4]

p_1 = a = 9
p_2 = b = 1
p_3 = c = 10
p_4 = d = 2
p_5 = e = 11
p_6 = f = 4

Spots Above 0's

s(1,2) -> a * b * c = 9 * 1 * 10 = 90
s(2,3) -> b *c * d = 1 * 10 * 2 = 20
s(3,4) -> c * d * e = 10 * 2 * 11 = 220
s(4,5) -> d * e * f = 2 * 11 * 4 = 88

Solving for other spaces

Formula Stuff

For space **s(i,j)**, **k = {i, i + 1, i + 2, …, j - 2, j - 1}**
For each value of k, do the following:
$s(i,j) = s(i,k) + s(k + 1,j) + p_i * p_{k+1} * p_{j+1}$
For the best case, **choose the smallest value.**
For the worst case, **choose the largest value. This one wants the worst case, so take this one.**
For the average case, **take the cases for the top-right square (in this case [1,5]) and take their average**

s(1,3); k = 1, 2
k = 1 -> s(1,1) + s(2,3) + $p_1 * p_2 * p_4$ = 0 + 20 + 9 * 1 * 2 = 20 + 18 = 38
k = 2 -> s(1,2) + s(3,3) + $p_1 * p_3 * p_4$ = 90 + 0 + 9 * 10 * 2 = 90 + 180 = **270** (largest!)

s(2,4); k = 2, 3
k = 2 -> s(2,2) + s(3,4) + $p_2 * p_3 * p_5$ = 0 + 220 + 1 * 10 * 11 = 220 + 110 = **330** (largest!)
k = 3 -> s(2,3) + s(4,4) + $p_2 * p_4 * p_5$ = 20 + 0 + 1 * 2 * 11 = 20 + 22 = 44

s(3,5); k = 3, 4
k = 3 -> s(3,3) + s(4,5) + $p_3 * p_4 * p_6$ = 0 + 88 + 10 * 2 * 4 = 88 + 80 = 160
k = 4 -> s(3,4) + s(5,5) + $p_3 * p_5 * p_6$ = 220 + 0 + 10 * 11 * 4 = 220 + 440 = **660** (largest!)

s(1,4); k = 1, 2, 3
k = 1 -> s(1,1) + s(2,4) + $p_1 * p_2 * p_5$ = 0 + 330 + 9 * 1 * 11 = 330 + 99 = 429
k = 2 -> s(1,2) + s(3,4) + $p_1 * p_3 * p_5$ = 90 + 220 + 9 * 10 * 11 = 310 + 990 = **1300** (largest!)
k = 3 -> s(1,3) + s(4,4) + $p_1 * p_4 * p_5$ = 270 + 0 + 9 * 2 * 11 = 270 + 198 = 468

s(2,5); k = 2, 3, 4
k = 2 -> s(2,2) + s(3,5) + p_2 * p_3 * p_6 = 0 + 660 + 1 * 10 * 4 = 660 + 40 = **700** (largest!)
k = 3 -> s(2,3) + s(4,5) + p_2 * p_4 * p_6 = 20 + 88 + 1 * 2 * 4 = 108 + 8 = 116
k = 4 -> s(2,4) + s(5,5) + p_2 * p_5 * p_6 = 330 + 0 + 1 * 11 * 4 = 330 + 44 = 374

s(1,5); k = 1, 2, 3, 4
k = 1 -> s(1,1) + s(2,5) + p_1 * p_2 * p_6 = 0 + 700 + 9 * 1 * 4 = 700 + 36 = 736
k = 2 -> s(1,2) + s(3,5) + p_1 * p_3 * p_6 = 90 + 660 + 9 * 10 * 4 = 750 + 360 = 1110
k = 3 -> s(1,3) + s(4,5) + p_1 * p_4 * p_6 = 270 + 88 + 9 * 2 * 4 = 358 + 72 = 430
k = 4 -> s(1,4) + s(5,5) + p_1 * p_5 * p_6 = 1300 + 0 + 9 * 11 * 4 = 1300 + 396 = **1696** (answer!)

It might ask for average number of multiplications if so take the top right most square for example S[1,5] and add the test cases up and divide them up by the amout of cases present, for example 576+927+392+1251/4 = 786.5

Problem 4

⅗ times is a rectangular matrix problem, ⅖ times is a huffman code problem

2022

Construct a Huffman code for the symbols a through g, listed below together with their probabilities.
Then determine the expected length of your resulting code!

a - 1/45	b - 2/45	c - 3/45	d - 4/45	e - 5/45	f - 6/45	g - 10/45	h - 14/45

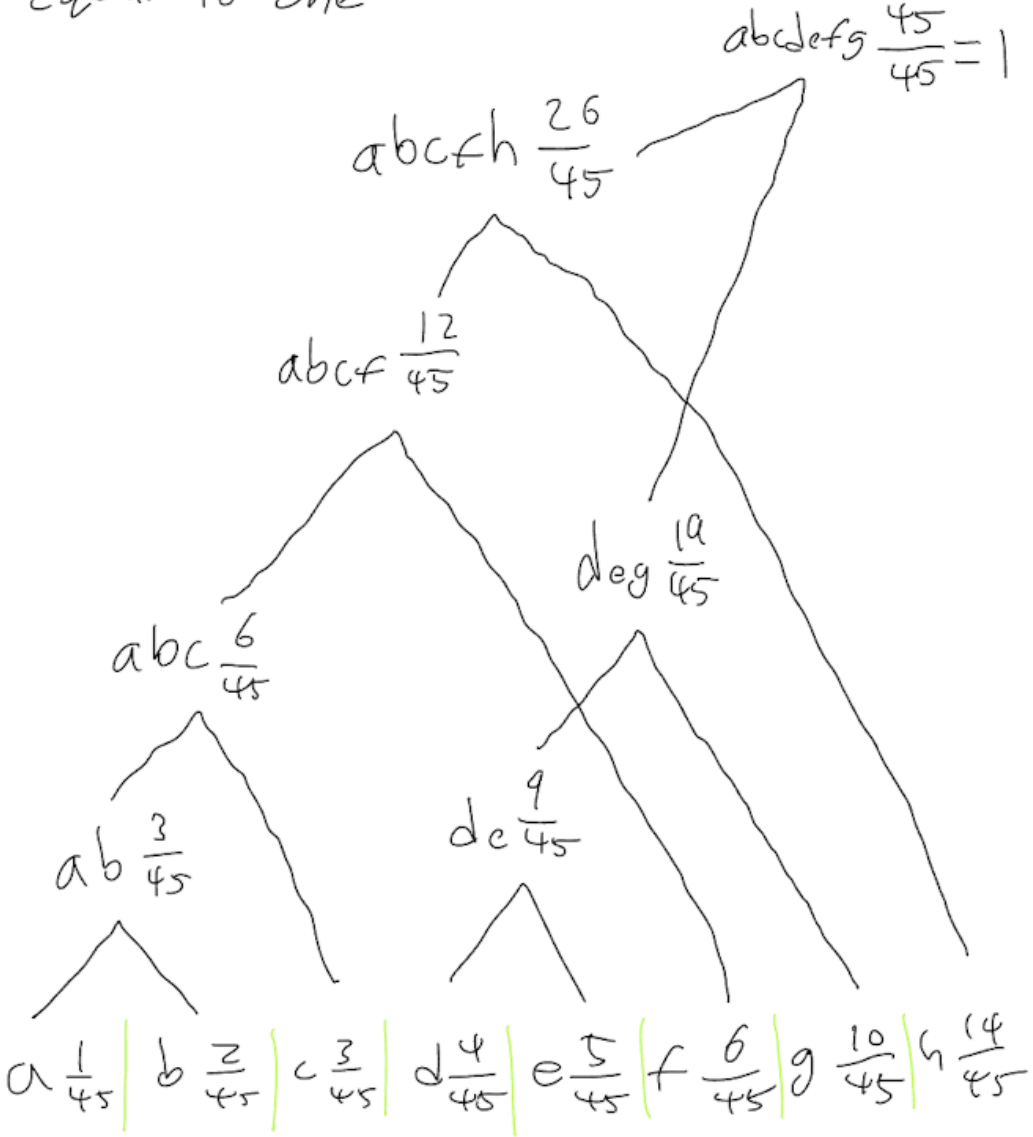
How to solve Huffman

Based on 2022 Question 4

Step 1 arrange all probabilities at bottom of page

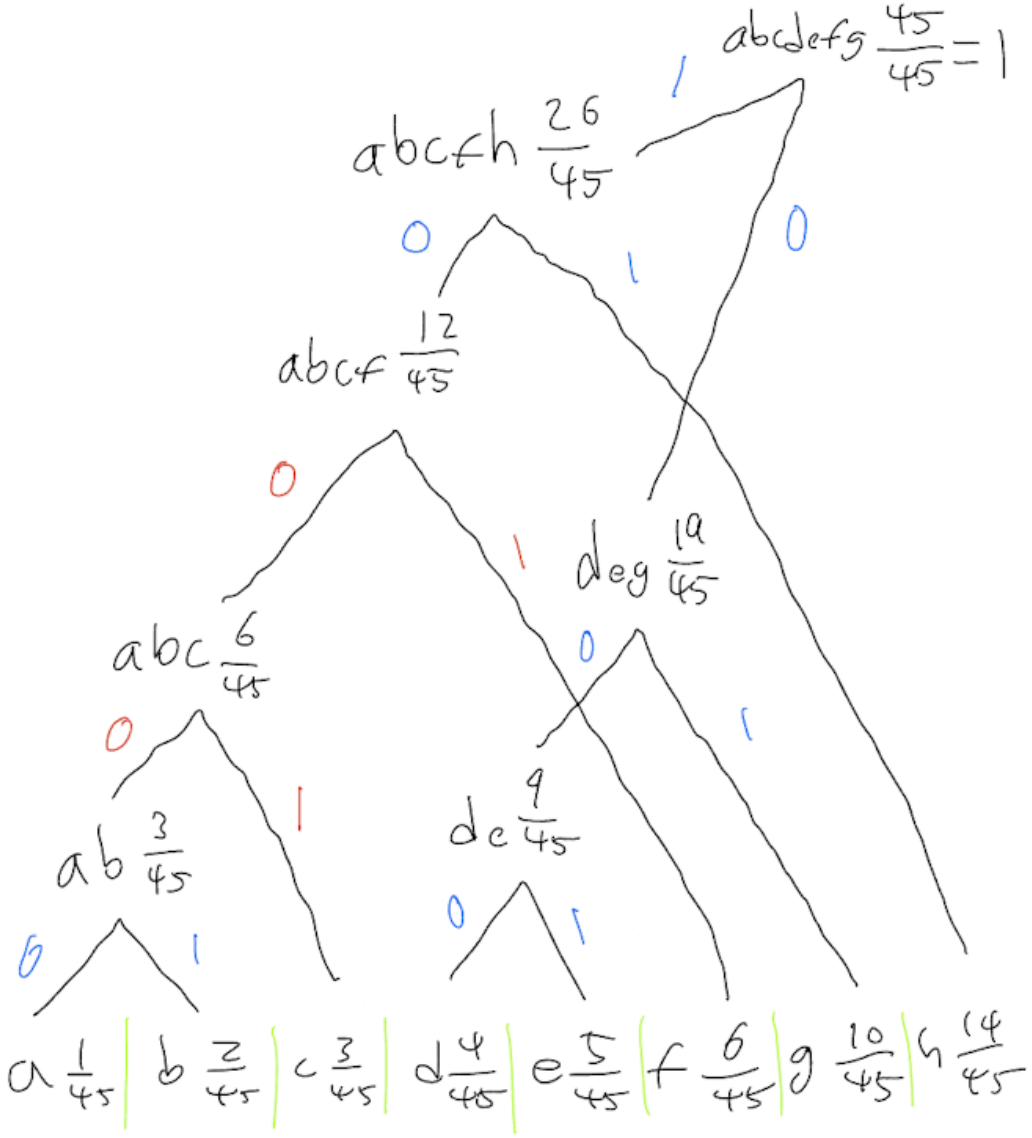
a $\frac{1}{45}$ b $\frac{2}{45}$ c $\frac{3}{45}$ d $\frac{4}{45}$ e $\frac{5}{45}$ f $\frac{6}{45}$ g $\frac{10}{45}$ h $\frac{14}{45}$

Step 2 merge the two smallest until the probability is equal to one



step 3 assign 1's and 0's from the root to each branch.

* assign a 0 to the branch with a smaller value, 1 to a bigger branch
* if the frequencies on each branch are equal, assign a 0 to the branch with more characters and 1 to the branch with less



Step 3 trace the Frequencies from the root to the bottom to assign codes

a 10000
b 10001
c 1001
d 000
e 001
f 101
g 01
h 11

* If you want to convert this to a nicer looking tree, arrange the branches from the root where the 0 branch is always on the left

Step 4 answer the question (B, C, H)

The expected length of huffman code is always $n \log n$ where n is equal to the number of characters.

This problem has 8 characters, so the expected length is $8 \log 8$

Problem 5
2/2 times is a heap problem

2020

(a) Construct a heap for the following array of numbers: 1 2 8 3 5 6 4 7 10 9
Show the array after the insertion of each element into the heap.
(b) Use your heap to sort the array. Show the resulting heap after the extraction of each maximum.

2022

Construct a heap for the following array of numbers:

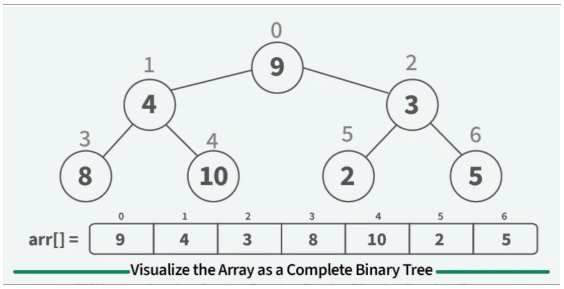
10	8	1	2	3	5	6	4	7	9

Show the array after the insertion of each element into the heap.
Use your heap to sort the array. Show the resulting heap after the extraction of each maximum.

A Really Badly Worded Heap Guide

General Notes

1. A heap is like a binary tree like in the example image below (except this one isn't sorted):



2. It has to follow the following rules:
    a. Each node has up to 2 children
    b. No child node's value can be greater than its parent node's value
    c. For layer n, there are 2^(n - 1) nodes in that layer; there's 1 in layer 1, 2 in layer 2, 4 in layer 3, etc
3. If the second rule is violated, swap the position of the parent and child node in both the array and the tree

Heap Creation

1. Insert the values from the array in order from first to last
2. The 1st element is the root node of the tree, the 2nd and 3rd are the child nodes of node 1, the 4th and 5th are children of node 2, the 6th and 7th are children of node 3, the 8th and 9th are children of node 4, the 10th and 11th are children of node 5, etc.
    a. **For a parent node whose value is at element n, its 2 child nodes' values would be at elements 2n and 2n + 1.**
3. After each insert, check whether the inserted node's value exceeds its parent's value; if yes, swap the positions of the insert and its parent in the array and the tree, then continue doing parent-child comparisons until no child node in the tree has a value that exceeds its parent's value. **This process is called heapifying.**

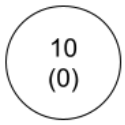
Heap Sort

1. Take the heap created from heap creation and the array made alongside it. Make a secondary empty array.
2. Swap the position of the root node element (the 1st element in the array) with the last element in the array (one of the leaf nodes).
3. Eliminate the leaf with the former root node element, then insert said element into the first element of the secondary array, pushing back any other present elements.
4. Heapify the remaining tree/array until no child nodes have values exceeding their parent nodes'.
5. Repeat steps 2 through 4 until the heap tree/array is only 1 element long, then remove that last element and push it to the secondary sorted array.

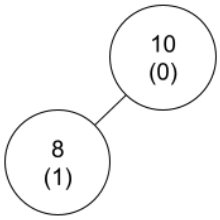
Seiun's Crappy Attempt to Solve (FEATURING DIAGRAMS)

Heap Creation

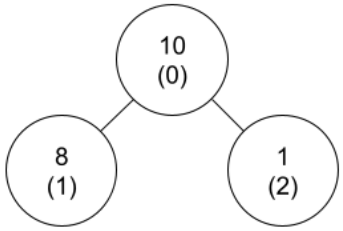
For heap sort, we need to make a max heap via the below steps.
Insert the first element: [10]. 10 takes the spot of element 0 in the image above, making it the base node.



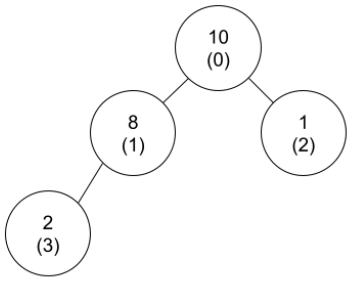
Insert the second element: [10, 8]. 8 takes the spot of element 1 in the image above, making it the left child node of 10.



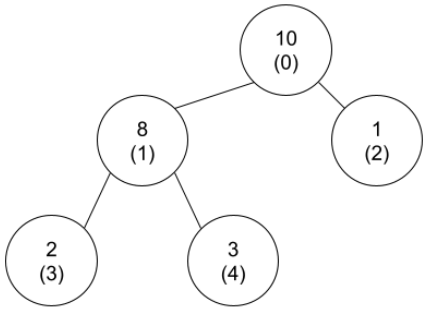
Insert the third element: [10, 8, 1]. 1 takes the spot of element 2 in the array, making it the right child node of 10.



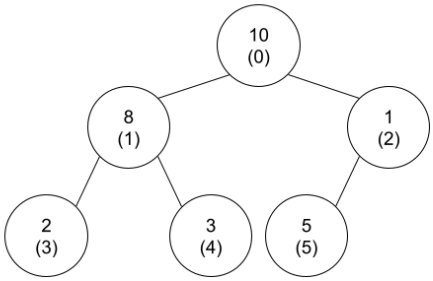
Insert the 4th element: [10, 8, 1, 2]. 2 takes the spot of element 3 in the array, making it the left child node of 8.



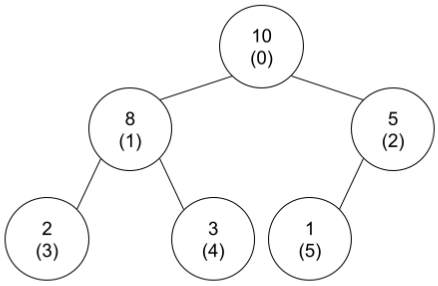
Insert the 5th element: [10, 8, 1, 2, 3]. 3 takes the spot of element 4 in the array, making it the right child node of 8.



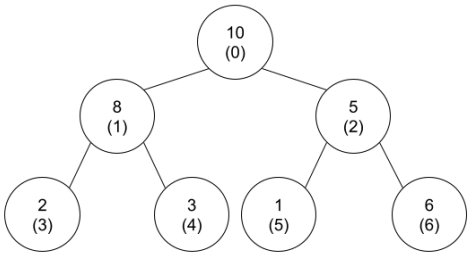
Insert the 6th element: [10, 8, 1, 2, 3, 5]. 5 takes the spot of element 5 in the array, making it the left child node of 1.



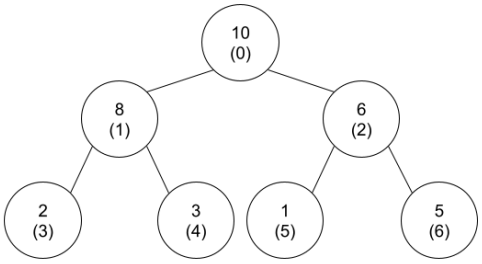
BUT WAIT! A max heap's nodes can't have a child that's larger than its parent, and 5 > 1! Thus, swap the positions of 1 and 5: [10, 8, 5, 2, 3, 1]. 1 is now the left child of 5, and 5 is now the right child of 10.



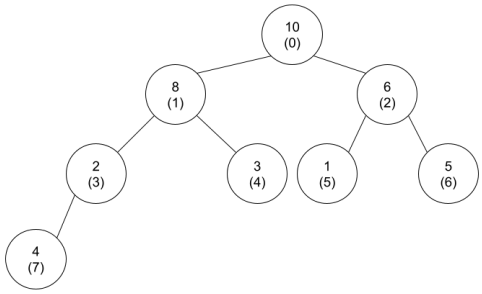
Insert the 7th element: [10, 8, 5, 2, 3, 1, 6]. 6 takes the spot of element 6, making it the right child of 5.



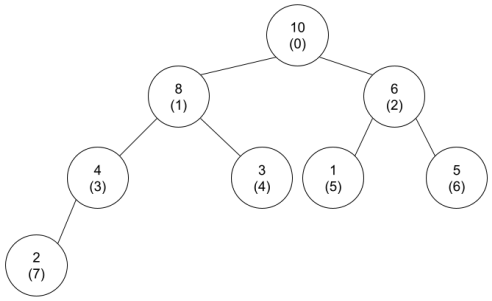
BUT WAIT! 6 > its parent, 5! Swap the positions of 6 and 5: [10, 8, 6, 2, 3, 1, 5]. 6 is now the right child of 10, and 5 is now the right child of 6.



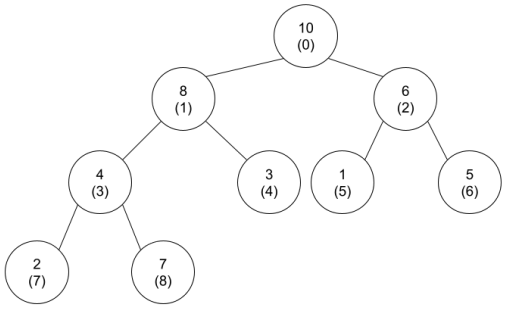
Insert the 8th element: [10, 8, 6, 2, 3, 1, 5, 4]. 4 would be in the spot of a hypothetical element 7, so it'd be the left child of 2.



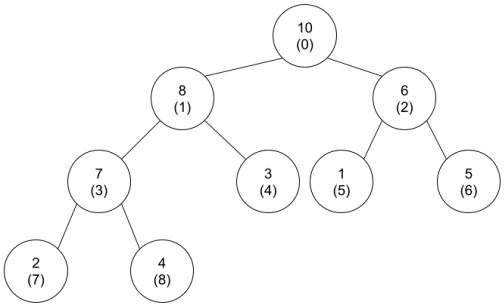
BUT WAIT! 4 > its parent, 2! Swap the positions of 2 and 4: [10, 8, 6, 4, 3, 1, 5, 2]



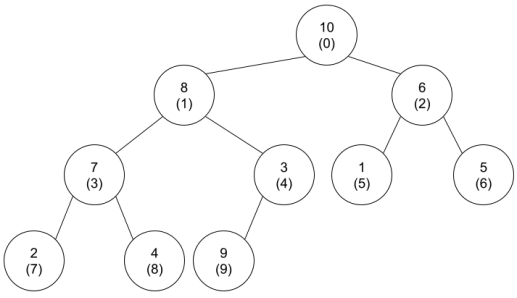
Insert the 9th element: [10, 8, 6, 4, 3, 1, 5, 2, 7]. 7 would be in the spot of a hypothetical element 8, so it'd be the right child of 4.



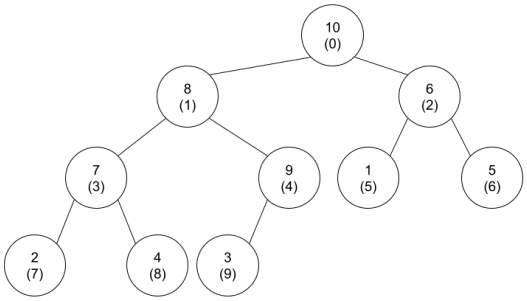
BUT WAIT! 7 > its parent, 4! Swap the positions of 7 and 4: [10, 8, 6, 7, 3, 1, 5, 2, 4].



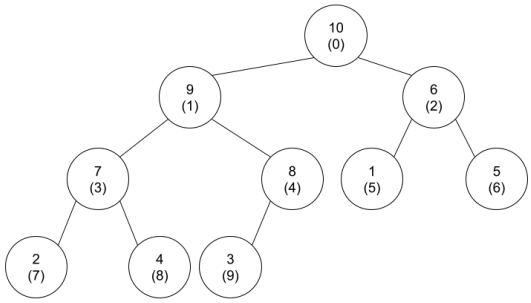
Insert the 10th element: [10, 8, 6, 7, 3, 1, 5, 2, 4, 9]. 9 would be in the spot of a hypothetical element 9, so it'd be the left child of 3.



BUT WAIT! 9 > its parent, 3! Swap the positions of 9 and 3: [10, 8, 6, 7, 9, 1, 5, 2, 4, 3]



BUT WAIT AGAIN! Now 9 > its new parent, 8! Swap the positions of 9 and 8: [10, 9, 6, 7, 8, 1, 5, 2, 4, 3]



This should be the final heap (I think; image is above) = **[10, 9, 6, 7, 8, 1, 5, 2, 4, 3]**

Sorting:

Max heap from the previous part:

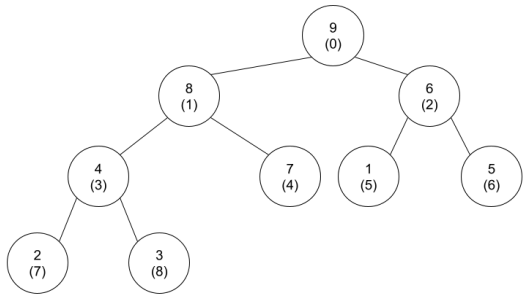
10	9	6	7	8	1	5	2	4	3

To sort this, we need to do the following steps:

- Swap the root node (element 0) w/ the last element in the array/heap
- Remove the original root node and push it to the front of another array; **the question wants the heap after each maximum extraction, so I'll bold these arrays**
- Re-heapify the heap, swapping any child elements that are greater than their parents with said parents
- Repeat until the heap is only 1 element large

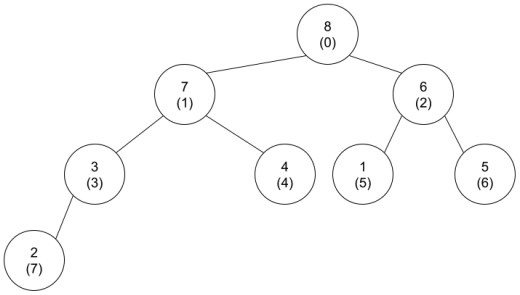
I'm not going to post images for every step because, well, god no, but I will post one after every single heapify.

Swap 10 (root) w/ 3 (last element): [3, 9, 6, 7, 8, 1, 5, 2, 4, 10]
Eliminate 10: **[3, 9, 6, 7, 8, 1, 5, 2, 4]; {10}**
9 > its parent, 3: swap 9 and 3: [9, 3, 6, 7, 8, 1, 5, 2, 4]; {10}
7 > its parent, 3: swap 7 and 3: [9, 7, 6, 3, 8, 1, 5, 2, 4]; {10}
8 > its parent, 7: swap 8 and 7: [9, 8, 6, 3, 7, 1, 5, 2, 4]; {10}
4 > its parent, 3: swap 4 and 3: [9, 7, 6, 4, 8, 1, 5, 2, 3]; {10}
Heapify done: [9, 8, 6, 4, 7, 1, 5, 2, 3]; {10}

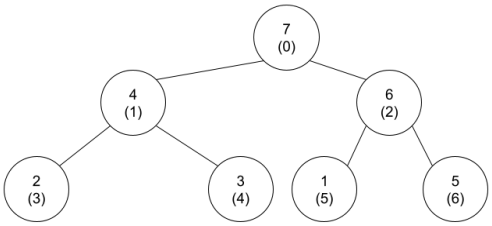


Swap 9 (root) w/ 3 (last element): [3, 8, 6, 4, 7, 1, 5, 2, 9]; {10}
Eliminate 9: **[3, 8, 6, 4, 7, 1, 5, 2]; {9, 10}**
8 > its parent, 3: swap 8 and 3: [8, 3, 6, 4, 7, 1, 5, 2]; {9, 10}

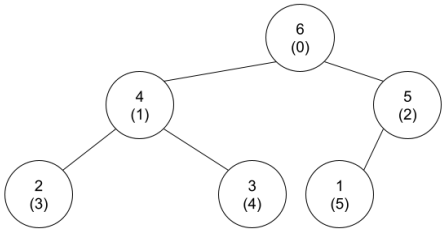
4 > its parent, 3: swap 4 and 3: [8, 4, 6, 3, 7, 1, 5, 2]; {9, 10}
7 > its parent, 4: swap 7 and 4: [8, 7, 6, 3, 4, 1, 5, 2]; {9, 10}
Heapify done: [8, 7, 6, 3, 4, 1, 5, 2]; {9, 10}



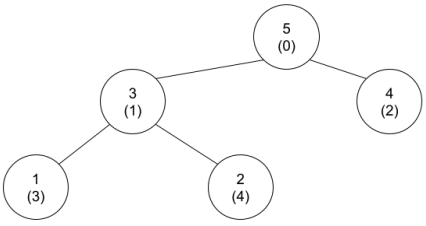
Swap 8 (root) w/ 2 (last element): [2, 7, 6, 3, 4, 1, 5, 8]; {9, 10}
Eliminate 8: **[2, 7, 6, 3, 4, 1, 5]; {8, 9, 10}**
7 > its parent, 2: swap 7 and 2: [7, 2, 6, 3, 4, 1, 5]; {8, 9, 10}
3 > its parent, 2: swap 3 and 2: [7, 3, 6, 2, 4, 1, 5]; {8, 9, 10}
4 > its parent, 3: swap 4 and 3: [7, 4, 6, 2, 3, 1, 5]; {8, 9, 10}
Heapify done: [7, 4, 6, 2, 3, 1, 5]; {8, 9, 10}



Swap 7 (root) w/ 5 (last element): [5, 4, 6, 2, 3, 1, 7]; {8, 9, 10}
Eliminate 7: **[5, 4, 6, 2, 3, 1]; {7, 8, 9, 10}**
6 > its parent, 5: swap 6 and 5: [6, 4, 5, 2, 3, 1]; {7, 8, 9, 10}
Heapify done: [6, 4, 5, 2, 3, 1]; {7, 8, 9, 10}

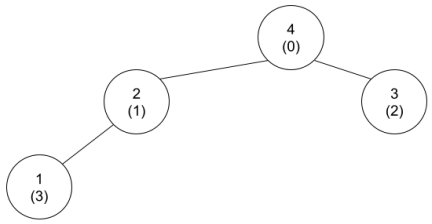


Swap 6 (root) w/ 1 (last element): [1, 4, 5, 2, 3, 6]; {7, 8, 9, 10}
Eliminate 6: **[1, 4, 5, 3, 2]; {6, 7, 8, 9, 10}**
4 > its parent, 1: swap 4 and 1: [4, 1, 5, 2, 3]; {6, 7, 8, 9, 10}
5 > its parent, 4: swap 5 and 4: [5, 1, 4, 2, 3]; {6, 7, 8, 9, 10}
2 > its parent, 1: swap 2 and 1: [5, 2, 4, 1, 3]; {6, 7, 8, 9, 10}
3 > its parent, 2: swap 3 and 2: [5, 3, 4, 1, 2]; {6, 7, 8, 9, 10}
Heapify done: [5, 3, 4, 1, 2]; {6, 7, 8, 9, 10}

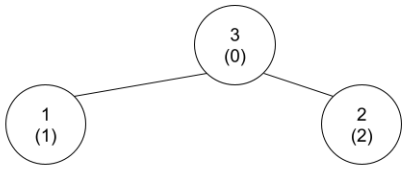


Swap 5 (root) w/ 2 (last element): [2, 3, 4, 1, 5]; {6, 7, 8, 9, 10}
Eliminate 5: **[2, 3, 4, 1]; {5, 6, 7, 8, 9, 10}**

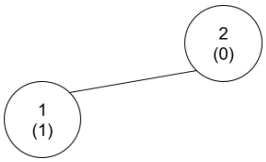
3 > its parent, 2: swap 3 and 2: [3, 2, 4, 1]; {5, 6, 7, 8, 9, 10}
4 > its parent, 3: swap 4 and 3: [4, 2, 3, 1]; {5, 6, 7, 8, 9, 10}
Heapify done: [4, 2, 3, 1]; {5, 6, 7, 8, 9, 10}



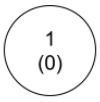
Swap 4 (root) w/ 1 (last element): [1, 2, 3, 4]; {5, 6, 7, 8, 9, 10}
Eliminate 4: **[1, 2, 3]; {4, 5, 6, 7, 8, 9, 10}**
2 > its parent, 1: swap 2 and 1: [2, 1, 3]; {4, 5, 6, 7, 8, 9, 10}
3 > its parent, 2: swap 3 and 2: [3, 1, 2]; {4, 5, 6, 7, 8, 9, 10}
Heapify done: [3, 1, 2]; {4, 5, 6, 7, 8, 9, 10}



Swap 3 (root) w/ 2 (last element): [2, 1, 3]; {4, 5, 6, 7, 8, 9, 10}
Eliminate 3: **[2, 1]; {3, 4, 5, 6, 7, 8, 9, 10}**
Heapify done: [2, 1]; {3, 4, 5, 6, 7, 8, 9, 10}



Swap 2 (root) w/ 1 (last element): [1, 2]; {3, 4, 5, 6, 7, 8, 9, 10}
Eliminate 2: **[1]; {2, 3, 4, 5, 6, 7, 8, 9, 10}**
Heap is of size 1! Eliminate 1 to get the sorted array.



Sorted array: **{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}**
Stick a fork in it, you're done!

(a) Construct a heap for the following array of numbers: 1 2 8 3 5 6 4 7 10 9
Show the array after the insertion of each element into the heap.

6. Determine the bit complexity of en- and decryption in RSA (M^e mod n, C^d mod n) with n having s digits, using the multiplication algorithm covered in class.

RSA en- and decryption

Determine the bit complexity of en- and decryption in RSA (M^e mod n, C^d mod n) with n having s digits, using the multiplication algorithm covered in class.

Using the multiplication algorithm covered in class, we will have 3 recursive calls instead of the traditional 4. For 'S' length digits, this method gives bit complexity of:

$O(s^{\log_2 3})$ or $O(s^{1.59})$ - (i)

Also, the multiplication algorithm for any number in form a^m gives us no more than $2\log_2 m$ - (ii) multiplication at most.

So, using values of i and ii, the total complexity is,

$O(2\log_2 n \cdot s^{\log_2 3})$